
Group B

Assignment No: 1

Aim: Write a code in JAVA for a simple WordCount application that counts the number of occurrences of each word in a given input set using the Hadoop MapReduce framework on local-standalone set-up.

Pre-requisite:

- i) Java Installation
- ii) Hadoop Installation

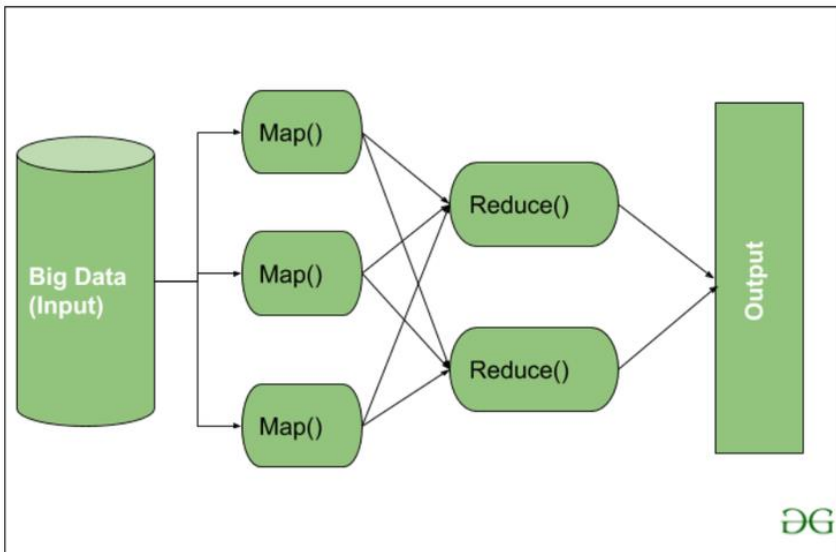
Theory:

Hadoop:

Hadoop is a framework written in [Java](#) programming language that works over the collection of commodity hardware. Before Hadoop, we are using a single system for storing and processing data. Also, we are dependent on RDBMS which only stores the structured data. To solve the problem of such huge complex data, Hadoop provides the best solution.

Map Reduce in Hadoop :

One of the three components of Hadoop is Map Reduce. The first component of Hadoop that is, Hadoop Distributed File System (HDFS) is responsible for storing the file. The second component that is, Map Reduce is responsible for processing the file. MapReduce has mainly 2 tasks which are divided phase-wise. In first phase, Map is utilised and in next phase Reduce is utilised.



Wordcount Program steps

Step 1) mkdir words

Step 2) Download hadoop-core-1.2.1.jar, which is used to compile and execute the MapReduce program. Visit the following

link

<http://mvnrepository.com/artifact/org.apache.hadoop/hadoop-core/1.2.1>

Step 3) Put that downloaded jar file into words folder.

Step 4) Implement WordCount.java program.

Step 5) Create input1.txt on home directory with some random text

Step 6) go on words path then compile

```
javac -classpath /home/vijay/words/hadoop-core-1.2.1.jar /home/vijay/words/WordCount.java
```

```
javac -classpath $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-client-core-3.2.4.jar:$HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-client-common-3.2.4.jar:$HADOOP_HOME/share/hadoop/common/hadoop-common-3.2.4.jar /home/gurukul/WordCount.java
```

Step 7) jar -cvf words.jar -c words/ .

Step 8) cd .. then use following commands

```
hadoop fs -mkdir /input
```

```
hadoop fs -put input1.txt /input
```

```
hadoop fs -ls /input
```

```
hadoop jar /home/vijay/words/words12.jar WordCount /input/input1.txt /out321
```

```
hadoop fs -ls /out321
```

```
hadoop fs -cat /out321/part-r-00000
```

(Otherwise check in Browsing HDFS -> Utilities -> Browse the file System -> /)

Assignment Questions

- 1. What is the map reduce explain with a small example?**
- 2. Write down steps to install hadoop.**

Group B

Assignment No: 2

-----Aim: Design a distributed application using MapReduce which processes a log file of a system

Theory:

- **Steps to Install Hadoop for distributed environment**
- **Java Code for processes a log file of a system**

Steps to Install Hadoop for distributed environment:

Initially create one folder logfiles1 on desktop. In that folder store input file (access_log_short.csv), SalesMapper.java, SalesCountryReducer.java, SalesCountryDriver.java files)

Step 1) Go to Hadoop home directory and format the NameNode.

```
cd hadoop-2.7.3
```

```
bin/hadoop namenode -format
```

Step 2) Once the NameNode is formatted, go to hadoop-2.7.3/sbin directory and start all the daemons/nodes.

```
cd hadoop-2.7.3/sbin
```

1) Start NameNode:

The NameNode is the centerpiece of an HDFS file system. It keeps the directory tree of all files stored in the HDFS and tracks all the file stored across the cluster.

```
./hadoop-daemon.sh start namenode
```

2) Start DataNode:

On startup, a DataNode connects to the Namenode and it responds to the requests from the Namenode for different operations.

```
./hadoop-daemon.sh start datanode
```

3) Start ResourceManager:

ResourceManager is the master that arbitrates all the available cluster resources and thus helps in managing the distributed applications running on the YARN system. Its work is to manage each NodeManagers and the each application's ApplicationMaster.

```
./yarn-daemon.sh start resourcemanager
```

4) Start NodeManager:

The NodeManager in each machine framework is the agent which is responsible for managing containers, monitoring their resource usage and reporting the same to the ResourceManager.

```
./yarn-daemon.sh start nodemanager
```

5) Start JobHistoryServer:

JobHistoryServer is responsible for servicing all job history related requests from client.

```
./mr-jobhistory-daemon.sh start historyserver
```

Step 3) To check that all the Hadoop services are up and running, run the below command.

jps

Step 4) cd

Step 5) sudo mkdir **mapreduce_vijay**

Step 6) sudo chmod 777 -R **mapreduce_vijay/**

Step 7) sudo chown -R **vijay mapreduce_vijay/**

Step 8) sudo cp /home/**vijay**/Desktop/logfiles1/* ~/**mapreduce_vijay/**

Step 9) cd **mapreduce_vijay/**

Step 10) ls

Step 11) sudo chmod +r *.*

Step 12) export CLASSPATH="/home/**vijay**/hadoop-2.7.3/share/hadoop/mapreduce/hadoop-mapreduce-client-core-2.7.3.jar:/home/**vijay**/hadoop-2.7.3/share/hadoop/mapreduce/hadoop-mapreduce-client-common-2.7.3.jar:/home/**vijay**/hadoop-2.7.3/share/hadoop/common/hadoop-common-2.7.3.jar:~/**mapreduce_vijay**/SalesCountry/*:\$HADOOP_HOME/lib/*"

Step 13) javac -d . SalesMapper.java SalesCountryReducer.java
SalesCountryDriver.java

Step 14) ls

Step 15) cd SalesCountry/

Step 16) ls (check is class files are created)

Step 17) cd ..

Step 18) gedit Manifest.txt

(add following lines to it:

Main-Class: SalesCountry.SalesCountryDriver)

Step 19) jar -cfm mapreduce_vijay.jar Manifest.txt SalesCountry/*.class

Step 20) ls

Step 21) cd

Step 22) cd mapreduce_vijay/

Step 23) sudo mkdir /input200

Step 24) sudo cp access_log_short.csv /input200

Step 25) \$HADOOP_HOME/bin/hdfs dfs -put /input200 /

Step 26) \$HADOOP_HOME/bin/hadoop jar mapreduce_vijay.jar /input200 /output200

Step 27) hadoop fs -ls /output200

Step 28) hadoop fs -cat /out321/part-00000

Step 29) Now open the Mozilla browser and go to **localhost:50070/dfshealth.html** to check the NameNode interface.

Assignment Questions

1. Write down the steps for Design a distributed application using MapReduce which processes a log file of a system.

Group B

Assignment No: 3

Theory:

- **Steps to Install Scala**
- **Apache Spark Framework Installation**
- **Souce Code**

1) Install Scala

Step 1) java -version

Step 2) Install **Scala** from the apt repository by running the following commands to search for scala and install it.

sudo apt search scala ⇒ Search for the package

sudo apt install scala ⇒ Install the package

Step 3) To verify the installation of **Scala**, run the following command.

scala -version

2) Apache Spark Framework Installation

Apache Spark is an open-source, distributed processing system used for **big data workloads**. It utilizes in-memory caching, and optimized query execution for fast analytic queries against data of any size.

Step 1) Now go to the official Apache Spark download page and grab the latest version (i.e. 3.2.1) at the time of writing this article. Alternatively, you can use the `wget` command to download the file directly in the terminal.

```
wget https://apachemirror.wuchna.com/spark/spark-3.2.1/spark-3.2.1-bin-hadoop2.7.tgz
```

Step 2) Extract the Apache Spark tar file.

```
tar -xvzf spark-3.1.1-bin-hadoop2.7.tgz
```

Step 3) Move the extracted **Spark** directory to **/opt** directory.

```
sudo mv spark-3.1.1-bin-hadoop2.7 /opt/spark
```

Configure Environmental Variables for Spark

Step 4) Now you have to set a few environmental variables in **.profile** file before starting up the spark.

```
echo "export SPARK_HOME=/opt/spark" >> ~/.profile
```

```
echo "export PATH=$PATH:/opt/spark/bin:/opt/spark/sbin" >> ~/.profile
```

```
echo "export PYSPARK_PYTHON=/usr/bin/python3" >> ~/.profile
```

Step 5) To make sure that these new environment variables are reachable within the shell and available to Apache Spark, it is also mandatory to run the following command to take recent changes into effect.

```
source ~/.profile
```

Step 6) `ls -l /opt/spark`

Start Apache Spark in Ubuntu

Step 7) Run the following command to start the **Spark** master service and slave service.

```
start-master.sh
```

```
start-workers.sh spark://localhost:7077
```

(if workers not starting then remove and install openssh:

```
sudo apt-get remove openssh-client openssh-server
```

```
sudo apt-get install openssh-client openssh-server)
```

Step 8) Once the service is started go to the browser and type the following URL access spark page. From the page, you can see my master and slave service is started.

<http://localhost:8080/>

URL: spark://LinuxShellTips:7077
 Alive Workers: 1
 Cores in use: 1 Total, 0 Used
 Memory in use: 5.6 GiB Total, 0.0 B Used
 Resources in use:
 Applications: 0 Running, 0 Completed
 Drivers: 0 Running, 0 Completed
 Status: ALIVE

▼ Workers (1)

Worker Id	Address	State	Cores	Memory	Resources
worker-20210501104244-192.168.1.5-39895	192.168.1.5:39895	ALIVE	1 (0 Used)	5.6 GiB (0.0 B Used)	

▼ Running Applications (0)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
----------------	------	-------	---------------------	------------------------	----------------	------	-------	----------

▼ Completed Applications (0)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
----------------	------	-------	---------------------	------------------------	----------------	------	-------	----------

Step 9) You can also check if **spark-shell** works fine by launching the **spark-shell** command.

Spark-shell

```
sudo apt install snapd
```

```
snap find "intellij"
```

```
sudo snap install intellij-idea-community - - classic
```

Start IntelliJ IDE community Edition

Source Code:

/* Sample Code to print Statement */

```
object ExampleString {  
    def main(args: Array[String]) {  
  
        //declare and assign string variable "text"  
        val text : String = "You are reading SCALA programming language.";  
  
        //print the value of string variable "text"  
  
        println("Value of text is: " + text);  
  
    }  
}
```

/Scala program to find a number is positive, negative or positive.**/**

```
object ExCheckNumber {  
    def main(args: Array[String]) {  
  
        /**declare a variable*/  
        var number= (-100);  
  
        if(number==0){  
            println("number is zero");  
        }  
        else if(number>0){
```

```
        println("number is positive");
    }
    else{
        println("number is negative");
    }
}
}
```

/*Scala program to print your name*/

```
object ExPrintName {
    def main(args: Array[String]) {
        println("My name is Mike!")
    }
}
```

/Scala Program to find largest number among two numbers.**/**

```
object ExFindLargest {
    def main(args: Array[String]) {
        var number1=20;
        var number2=30;
        var x = 10;

        if( number1>number2){
            println("Largest number is:" + number1);
        }
        else{
            println("Largest number is:" + number2);
        }
    }
}
```

Assignment Questions

- 1. Write down steps to install scala.**
- 2. What is Scala?**